



UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Departamento de Ciencias de la Computación
Asignatura: Desarrollo de Aplicaciones Móviles
NRC: 30738

Informe 2.1

Desarrollo de Calculadora Funcional en FlutterFlow

Integrantes:

Arroyo Alfonso
Cáceres Germán
Frías Pedro
Llumiquirea Ariel

Docente: Ing. Doris Chicacza MSc

Fecha: 19 de mayo de 2026

Link Entregables:

<https://github.com/Gpcaceres/grupo-moviles-poryectos.git>

Índice

Índice de Anexos	ii
1 Introducción	1
1.1 Objetivos	1
1.1.1 Objetivo General	1
1.1.2 Objetivos Específicos	1
2 Marco Teórico	1
2.1 FlutterFlow	1
2.2 Paradigma Low-Code	2
2.3 Gestión de Estado en FlutterFlow	2
2.4 Action Flow Editor	2
2.5 Inline Functions y Code Expressions	3
3 Arquitectura de la Calculadora en FlutterFlow	3
4 Procedimiento de Desarrollo	4
4.1 Paso 1 – Diseño de la Interfaz de Usuario	4
4.2 Paso 2 – Creación de la Variable de Estado	5
4.3 Paso 3 – Configuración del Action Flow para las Operaciones	6
4.4 Paso 4 – Implementación de Inline Functions	7
4.5 Paso 5 – Pruebas de Funcionalidad	7
5 Análisis de Resultados	7
5.1 Resultados de las Operaciones	8
5.2 Evaluación de Componentes	10
5.3 Observaciones Técnicas	10
6 Conclusiones	10
7 Recomendaciones	11
Referencias	12
Anexos	13

Índice de figuras

1	Arquitectura de flujo de datos de la calculadora en FlutterFlow	3
2	Espacio de trabajo de FlutterFlow – vista del editor visual	4
3	Árbol de componentes (<i>Widget Tree</i>) de la calculadora	5
4	Configuración de la variable de estado result (tipo Double)	6
5	Configuración del <i>Action Flow</i> – asignación de variables y operación	7
6	Resultado de la suma: $7 + 9 = 16$	8
7	Resultado de la resta: $7 - 9 = -2$	8
8	Resultado de la multiplicación: $7 \times 9 = 63$	9
9	Resultado de la división: $7 \div 9 \approx 0,7778$	9

Índice de Anexos

1	Operación de suma: $7 + 9 = 16$	13
2	Operación de resta: $7 - 9 = -2$	13
3	Operación de multiplicación: $7 \times 9 = 63$	14
4	Operación de división: $7 \div 9 \approx 0,7778$	14

1. Introducción

El desarrollo de aplicaciones móviles ha experimentado una transformación significativa con la aparición de plataformas de desarrollo visual y *low-code*. **FlutterFlow**, lanzado en 2020, se ha consolidado como una de las herramientas más potentes en este paradigma, permitiendo crear aplicaciones Flutter completamente funcionales mediante una interfaz gráfica de arrastrar y soltar (*drag-and-drop*), sin necesidad de escribir código manual [1]. Este informe documenta el proceso de desarrollo de una **calculadora funcional** en FlutterFlow, implementando las cuatro operaciones aritméticas básicas (suma, resta, multiplicación y división). El proyecto integra componentes visuales (*widgets*), variables de estado (*Page State*) y flujos de acciones (*Action Flow Editor*) para demostrar la capacidad de FlutterFlow de gestionar lógica de negocio compleja mediante configuración visual.

1.1 Objetivos

1.1.1 Objetivo General

Desarrollar una aplicación de calculadora funcional en FlutterFlow que implemente operaciones aritméticas básicas mediante la configuración visual de componentes, variables de estado y flujos de acciones.

1.1.2 Objetivos Específicos

- Diseñar la interfaz de usuario con campos de entrada numéricos y botones de operación.
- Implementar una variable de estado tipo `Double` para almacenar y mostrar resultados.
- Configurar flujos de acciones (*Action Flows*) para cada operación aritmética.
- Utilizar expresiones personalizadas (*Inline Functions*) para ejecutar operaciones matemáticas.
- Validar el funcionamiento de la calculadora mediante pruebas de las cuatro operaciones básicas.

2. Marco Teórico

2.1 FlutterFlow

FlutterFlow es una plataforma de desarrollo visual *low-code* creada por ex-ingenieros de Google, lanzada oficialmente en 2020. Permite diseñar, construir y desplegar aplicaciones Flutter completas sin necesidad de escribir código manualmente, mediante una interfaz de arrastrar y soltar [1]. A diferencia de otros constructores visuales que generan código propietario o difícil de mantener, FlutterFlow produce código Flutter estándar y limpio que puede exportarse, modificarse y mantenerse en cualquier IDE tradicional [3].

Las características principales de FlutterFlow incluyen:

- **Editor visual de UI:** construcción de interfaces mediante *widgets* de Flutter arrastrables.
- **Sistema de navegación:** configuración visual de rutas y transiciones entre páginas.

- **Gestión de estado:** variables de ámbito de aplicación (*App State*) y de página (*Page State*).
- **Action Flow Editor:** editor visual de lógica de negocio sin código.
- **Integraciones nativas:** conexión con Firebase, Supabase, APIs REST y servicios externos.
- **Exportación de código:** descarga del proyecto Flutter completo para desarrollo híbrido [1].

2.2 Paradigma Low-Code

Las plataformas *low-code* permiten desarrollar aplicaciones completas mediante configuración visual, reduciendo drásticamente la cantidad de código escrito manualmente. Según Gartner, se estima que para 2026 más del 65 % del desarrollo de aplicaciones empresariales se realizará mediante plataformas *low-code* o *no-code* [4]. Este paradigma democratiza el desarrollo de software, permitiendo que diseñadores, analistas de negocio y desarrolladores colaboren en un mismo entorno visual [3].

FlutterFlow se clasifica como *low-code* porque, aunque la mayor parte del desarrollo es visual, permite insertar código Dart personalizado mediante *Custom Functions* y *Custom Actions* cuando se requiere lógica compleja no soportada nativamente [1].

2.3 Gestión de Estado en FlutterFlow

El **estado** de una aplicación representa los datos que cambian a lo largo del tiempo (valores de formularios, resultados de cálculos, datos descargados de APIs, etc.). FlutterFlow implementa dos niveles de estado [1]:

- **App State:** variables globales accesibles desde cualquier página de la aplicación. Persisten durante toda la sesión del usuario.
- **Page State:** variables locales a una página específica. Se destruyen al salir de la página y se recrean al entrar nuevamente.

Cada variable de estado tiene un tipo de dato definido (**String**, **Integer**, **Double**, **Boolean**, **List**, etc.) y puede tener un valor inicial. Los *widgets* pueden “escuchar” estas variables y actualizarse automáticamente cuando cambian, implementando el patrón reactivo de Flutter [2].

2.4 Action Flow Editor

El **Action Flow Editor** es el editor visual de lógica de FlutterFlow. Permite configurar secuencias de acciones que se ejecutan en respuesta a eventos de usuario (clics, deslizamientos, envíos de formularios, etc.) [1]. Las acciones disponibles incluyen:

- **Navegación:** abrir páginas, regresar, abrir diálogos.
- **Actualización de estado:** modificar variables de *App State* o *Page State*.
- **Lógica condicional:** ejecutar acciones solo si se cumple una condición.
- **Operaciones de backend:** llamadas a APIs, consultas a Firebase, autenticación.
- **Funciones personalizadas:** ejecutar código Dart mediante *Inline Functions* o *Custom Actions*.

El *Action Flow Editor* estructura las acciones en un flujo visual, permitiendo encadenar múltiples pasos de forma secuencial o condicional, facilitando la comprensión y el mantenimiento de la lógica de negocio sin necesidad de leer código [1].

2.5 Inline Functions y Code Expressions

Cuando la lógica requerida no puede configurarse completamente con acciones visuales, FlutterFlow permite insertar **expresiones de código Dart** directamente en el flujo de acciones. Estas expresiones, llamadas *Inline Functions* o *Code Expressions*, reciben variables de entrada (**var1**, **var2**, etc.) y retornan un valor calculado [1].

Por ejemplo, para dividir dos números capturados desde campos de texto, se utiliza la expresión:

```
var1 / var2
```

Listing 1: Expresión Dart para división en FlutterFlow

Esta capacidad permite implementar lógica matemática, manipulación de cadenas, conversiones de datos y validaciones personalizadas sin abandonar el editor visual [1].

3. Arquitectura de la Calculadora en FlutterFlow

La figura 1 ilustra la arquitectura de componentes de la calculadora desarrollada en FlutterFlow. El flujo de datos comienza con la captura de valores numéricos desde los campos de entrada, pasa por el procesamiento mediante *Inline Functions* ejecutadas desde los botones de operación, y finaliza con la actualización de la variable de estado **result** que alimenta el widget de visualización.

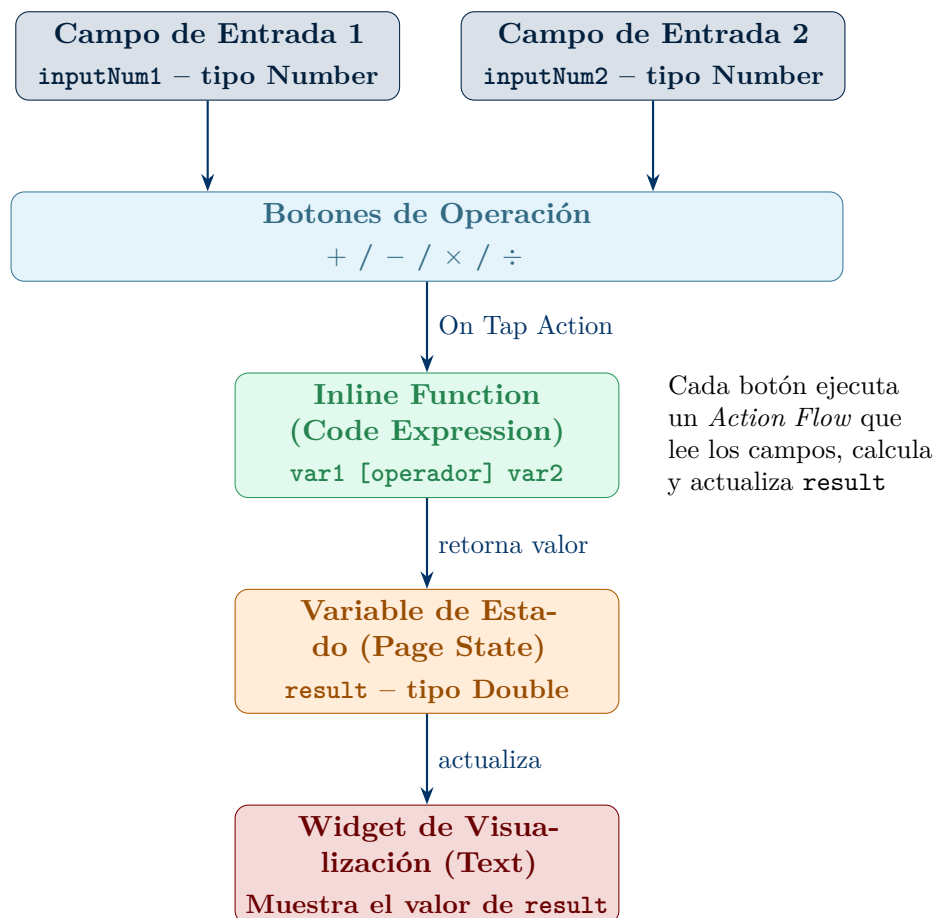


Figura 1: Arquitectura de flujo de datos de la calculadora en FlutterFlow

El espacio de trabajo de FlutterFlow (figura 2) muestra la interfaz completa del editor visual, incluyendo el árbol de *widets* a la izquierda, el lienzo de diseño al centro y el panel de propiedades a la derecha.

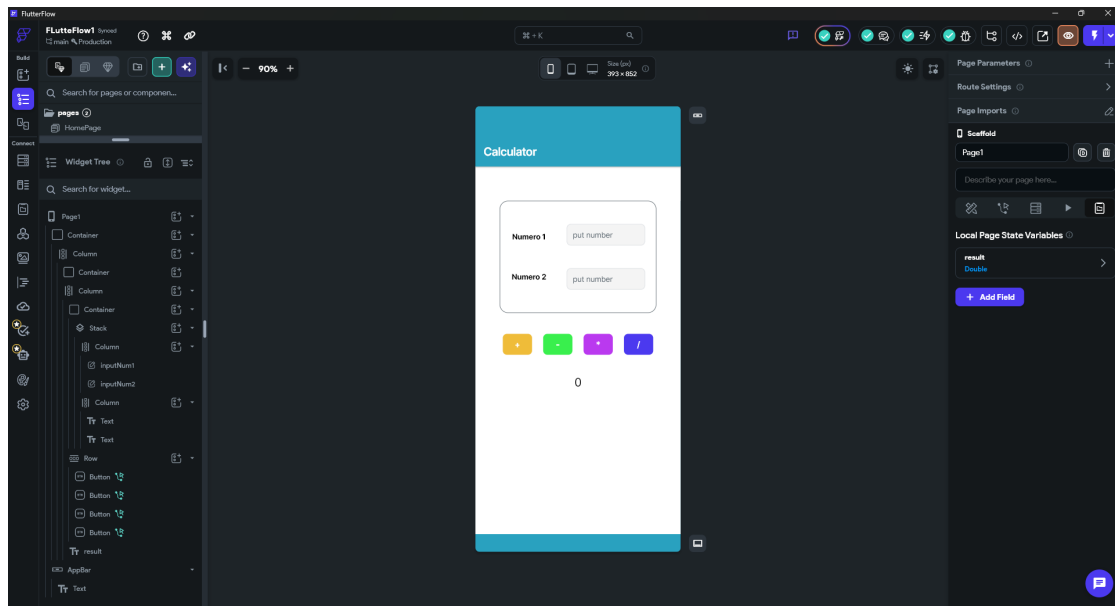


Figura 2: Espacio de trabajo de FlutterFlow – vista del editor visual

4. Procedimiento de Desarrollo

4.1 Paso 1 – Diseño de la Interfaz de Usuario

La interfaz de la calculadora se construyó utilizando los siguientes componentes de FlutterFlow:

- **Container:** contenedor principal que agrupa todos los elementos visuales.
- **Column:** organiza los elementos verticalmente (campos de entrada, botones, resultado).
- **TextField:** dos campos de entrada configurados para aceptar únicamente valores numéricos (`inputNum1` e `inputNum2`).
- **Row:** organiza los cuatro botones de operación horizontalmente.
- **Button:** cuatro botones (+, -, ×, ÷) con colores diferenciados.
- **Text:** widget de visualización del resultado en la parte inferior.

La figura 3 muestra la jerarquía de *widgets* utilizada en el proyecto.

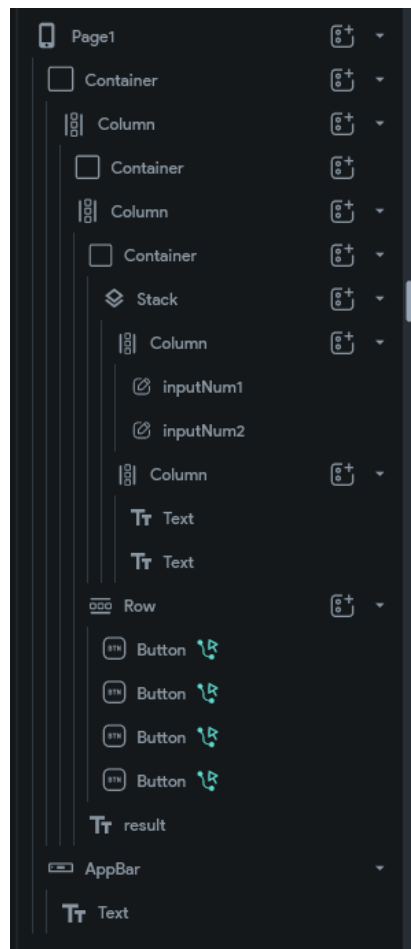


Figura 3: Árbol de componentes (*Widget Tree*) de la calculadora

Los campos de entrada fueron configurados con las siguientes propiedades:

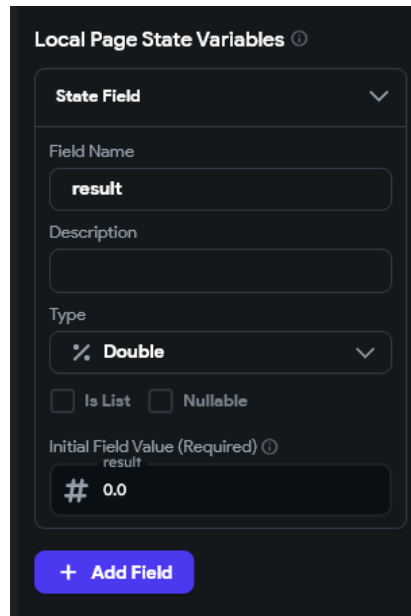
- **Keyboard Type:** *Number* (solo permite dígitos y punto decimal).
- **Hint Text:** “put number” (texto de ayuda).
- **Controller Name:** *inputNum1* e *inputNum2* para referencia posterior.

4.2 Paso 2 – Creación de la Variable de Estado

Para almacenar el resultado de las operaciones, se creó una variable de tipo **Page State** con las siguientes características:

- **Nombre:** *result*
- **Tipo:** *Double*
- **Valor inicial:** 0
- **Ámbito:** *Page State* (local a la página de la calculadora)

Las variables de *Page State* permiten que los *widgets* reaccionen automáticamente a cambios de valor. En este caso, el widget **Text** del resultado está vinculado a la variable *result*, actualizándose visualmente cada vez que esta cambia (figura 4).



Local Page State Variables ⓘ

State Field ▾

Field Name

result

Description

Type

Double ▾

☐ Is List ☐ Nullable

Initial Field Value (Required) ⓘ

result

0.0

+ Add Field

Figura 4: Configuración de la variable de estado `result` (tipo Double)

4.3 Paso 3 – Configuración del Action Flow para las Operaciones

Cada botón de operación (+, −, ×, ÷) ejecuta un **Action Flow** al ser presionado (evento `On Tap`). El flujo de acciones sigue la siguiente secuencia:

1. **Lectura de Valores:** se extraen los valores numéricos actuales de `inputNum1` e `inputNum2`.
2. **Ejecución de Operación:** se utiliza una *Inline Function* para realizar el cálculo matemático correspondiente.
3. **Actualización de Estado:** el resultado se almacena en la variable `result` mediante la acción `Update Page State`.
4. **Reconstrucción del Widget:** el widget `Text` vinculado a `result` se redibuja automáticamente con el nuevo valor.

La figura 5 muestra la configuración del *Action Flow Editor* para el botón de división, incluyendo la asignación de las dos variables de entrada (`var1` y `var2`) y la expresión Dart `var1 / var2`.

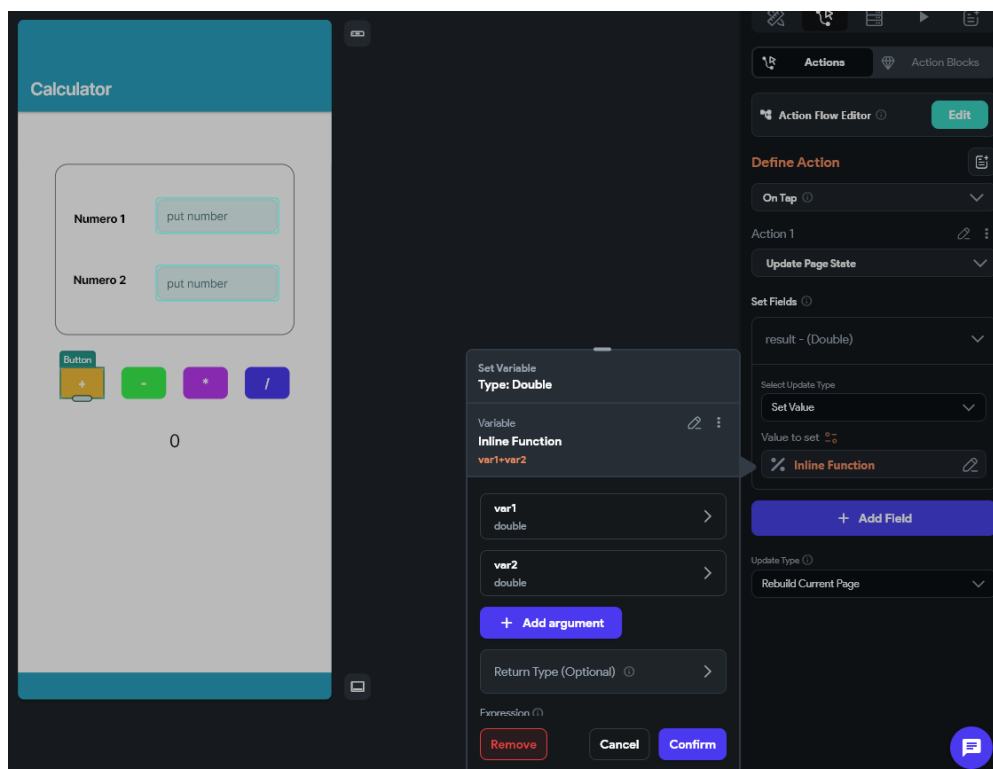


Figura 5: Configuración del *Action Flow* – asignación de variables y operación

4.4 Paso 4 – Implementación de Inline Functions

Para cada operación aritmética se configuró una **Inline Function** con los siguientes parámetros:

- **Variables de entrada:**
 - **var1** (tipo **double**): valor del primer campo de entrada.
 - **var2** (tipo **double**): valor del segundo campo de entrada.
- **Tipo de retorno:** **double**
- **Expresión de código:**
 - **Suma:** $\text{var1} + \text{var2}$
 - **Resta:** $\text{var1} - \text{var2}$
 - **Multiplicación:** $\text{var1} * \text{var2}$
 - **División:** $\text{var1} / \text{var2}$

Estas funciones se ejecutan en tiempo real al presionar cada botón, calculan el resultado y lo retornan a la acción **Update Page State** que actualiza la variable **result**.

4.5 Paso 5 – Pruebas de Funcionalidad

Se realizaron pruebas de cada operación con diferentes valores de entrada para validar el correcto funcionamiento de la calculadora. Las figuras 6 a 9 muestran los resultados de las pruebas realizadas con los valores de ejemplo 7 y 9.

5. Análisis de Resultados

La calculadora desarrollada en FlutterFlow demostró funcionar correctamente para las cuatro operaciones aritméticas básicas. Se realizaron pruebas con los valores de entrada

7 y **9**, obteniendo los resultados esperados en cada caso.

5.1 Resultados de las Operaciones

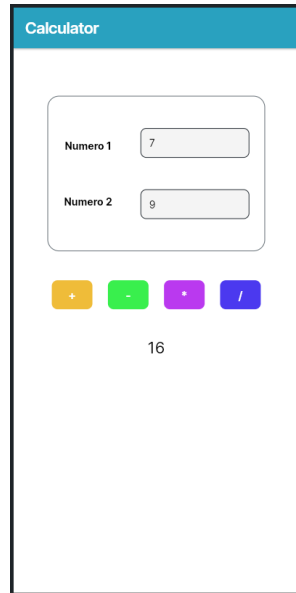


Figura 6: Resultado de la suma: $7 + 9 = 16$

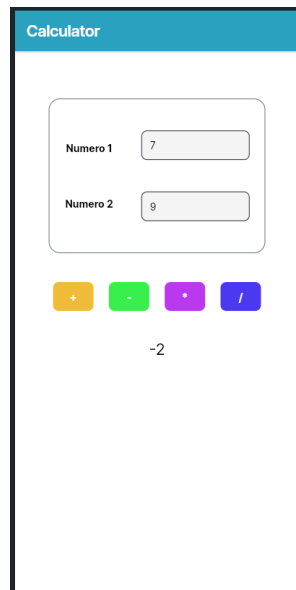


Figura 7: Resultado de la resta: $7 - 9 = -2$

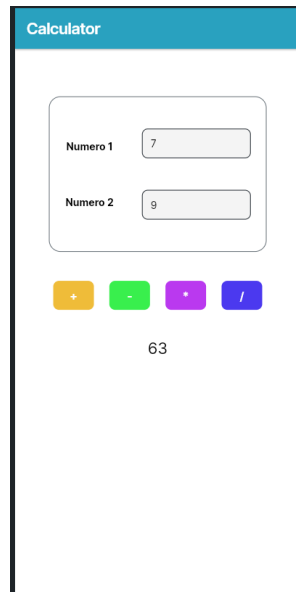


Figura 8: Resultado de la multiplicación: $7 \times 9 = 63$

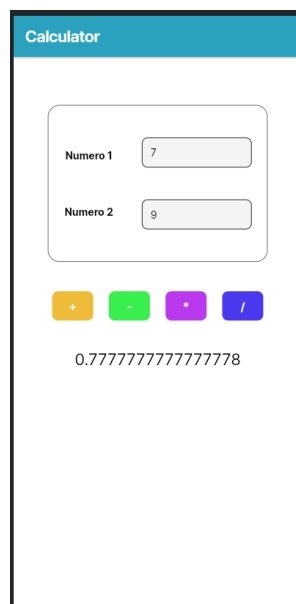


Figura 9: Resultado de la división: $7 \div 9 \approx 0,7778$

5.2 Evaluación de Componentes

La tabla 1 resume el estado de implementación de cada componente clave de la calculadora.

Cuadro 1: Estado de implementación de componentes de la calculadora

Componente	Estado	Funcionalidad
Campos de entrada numéricos	✓ Implementado	Captura valores correctamente
Botones de operación	✓ Implementado	4 operaciones disponibles
Variable de estado result	✓ Funcional	Almacena y actualiza resultados
Action Flow Editor	✓ Configurado	Flujos para cada operación
Inline Functions	✓ Implementado	Cálculos matemáticos precisos
Widget de visualización	✓ Reactivo	Actualización automática

5.3 Observaciones Técnicas

Durante el desarrollo se identificaron los siguientes aspectos relevantes:

1. **Actualización reactiva:** el widget `Text` vinculado a la variable `result` se actualiza automáticamente sin necesidad de código adicional, demostrando la naturaleza reactiva de `FlutterFlow`.
2. **Precisión decimal:** la división retorna valores de tipo `Double` con precisión decimal completa ($7 \div 9 = 0,777777\dots$), lo que confirma el correcto tipado de la variable de estado.
3. **Reutilización de flujos:** aunque cada botón tiene su propio *Action Flow*, la estructura es idéntica en todos los casos, variando únicamente el operador matemático en la *Inline Function*.
4. **Validación de entrada:** los campos `TextField` configurados con `Keyboard Type: Number` previenen automáticamente la entrada de caracteres no numéricos, mejorando la robustez de la aplicación.

6. Conclusiones

1. `FlutterFlow` permite desarrollar aplicaciones `Flutter` funcionales sin escribir código manualmente, mediante la combinación de componentes visuales (*widgets*), variables de estado y flujos de acciones configurables.
2. El sistema de **Page State** de `FlutterFlow` implementa el patrón reactivo de `Flutter`, permitiendo que los widgets se actualicen automáticamente cuando las variables a las que están vinculados cambian de valor, sin necesidad de gestionar manualmente la reconstrucción de la interfaz.

3. El **Action Flow Editor** proporciona una abstracción visual efectiva para implementar lógica de negocio, estructurando las operaciones en flujos secuenciales de acciones comprensibles sin necesidad de leer código Dart.
4. Las **Inline Functions** permiten integrar código Dart personalizado en puntos específicos del flujo, ofreciendo flexibilidad cuando la configuración visual no es suficiente, sin romper el flujo de trabajo visual predominante.
5. La configuración de tipos de datos en los componentes (**Number** para *TextFields*, **Double** para variables de estado) demuestra que FlutterFlow mantiene la seguridad de tipos característica de Dart, evitando errores en tiempo de ejecución.
6. El desarrollo de la calculadora evidencia que las plataformas *low-code* como FlutterFlow son apropiadas no solo para aplicaciones de visualización de datos, sino también para implementar lógica computacional, manejo de estado y flujos de interacción complejos.

7. Recomendaciones

1. **Planificar la estructura de variables de estado antes de implementar la interfaz.** Definir anticipadamente qué datos serán de ámbito *App State* (globales) y cuáles de *Page State* (locales) facilita el diseño de flujos de acciones y evita refactorizaciones posteriores.
2. **Utilizar nombres descriptivos para variables y componentes.** FlutterFlow genera referencias automáticas basadas en los nombres asignados (**inputNum1**, **result**, etc.); nombres claros mejoran la legibilidad de los *Action Flows* y facilitan el mantenimiento.
3. **Aprovechar la validación de tipos en TextFields.** Configurar el **Keyboard Type** apropiado (*Number*, *Email*, *Phone*) no solo mejora la experiencia de usuario, sino que previene errores de tipo en tiempo de ejecución al procesar los datos.
4. **Documentar las Inline Functions con comentarios.** Aunque FlutterFlow es visual, las expresiones de código Dart dentro de *Inline Functions* deben incluir comentarios explicativos cuando implementen lógica no trivial, facilitando la comprensión al exportar el proyecto.
5. **Probar los Action Flows con el modo de vista previa (Preview Mode).** FlutterFlow permite ejecutar la aplicación en tiempo real dentro del editor; utilizar esta funcionalidad reduce el tiempo de iteración al validar flujos de acciones sin necesidad de compilar.
6. **Considerar el manejo de errores en operaciones matemáticas.** Aunque la calculadora actual funciona correctamente con entradas válidas, implementar validaciones para casos límite (división por cero, campos vacíos, valores muy grandes) mediante condiciones en el *Action Flow* mejoraría la robustez de la aplicación.
7. **Explorar la exportación de código Flutter.** FlutterFlow permite descargar el proyecto como código Dart/Flutter estándar; revisar el código generado es una excelente oportunidad de aprendizaje para comprender cómo la configuración visual se traduce a código programático.
8. **Utilizar componentes reutilizables (Custom Widgets) para interfaces repetitivas.** Si la aplicación creciera con más funcionalidades (calculadora científica, conversiones de unidades, etc.), encapsular grupos de widgets en componentes personalizados facilitaría el mantenimiento y la consistencia visual.

Referencias

- [1] FlutterFlow Inc. (2024). *FlutterFlow Documentation – Build apps visually*. Recuperado de <https://docs.flutterflow.io/>
- [2] Google LLC. (2024). *Flutter – Build apps for any screen*. Flutter Documentation. Recuperado de <https://flutter.dev/docs>
- [3] Sahay, A., Indamutsa, A., Di Ruscio, D., & Pierantonio, A. (2023). *Supporting the collaborative development of low-code applications*. In Proceedings of the ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems.
- [4] Gartner, Inc. (2023). *Magic Quadrant for Enterprise Low-Code Application Platforms*. Gartner Research. Recuperado de <https://www.gartner.com/en/documents/4530099>
- [5] Google LLC. (2024). *Dart programming language – Overview*. Dart Documentation. Recuperado de <https://dart.dev/overview>
- [6] Richardson, C., & Rymer, J. R. (2020). *The Forrester Wave: Low-Code Development Platforms For Professional Developers, Q1 2020*. Forrester Research.
- [7] Windmill, E. (2020). *Flutter in Action*. Manning Publications. ISBN: 978-1-61729-571-8.
- [8] Statista Research. (2024). *Low-code development platform market revenue worldwide*. Recuperado de <https://www.statista.com/statistics/1224895/low-code-platform-market-revenue/>

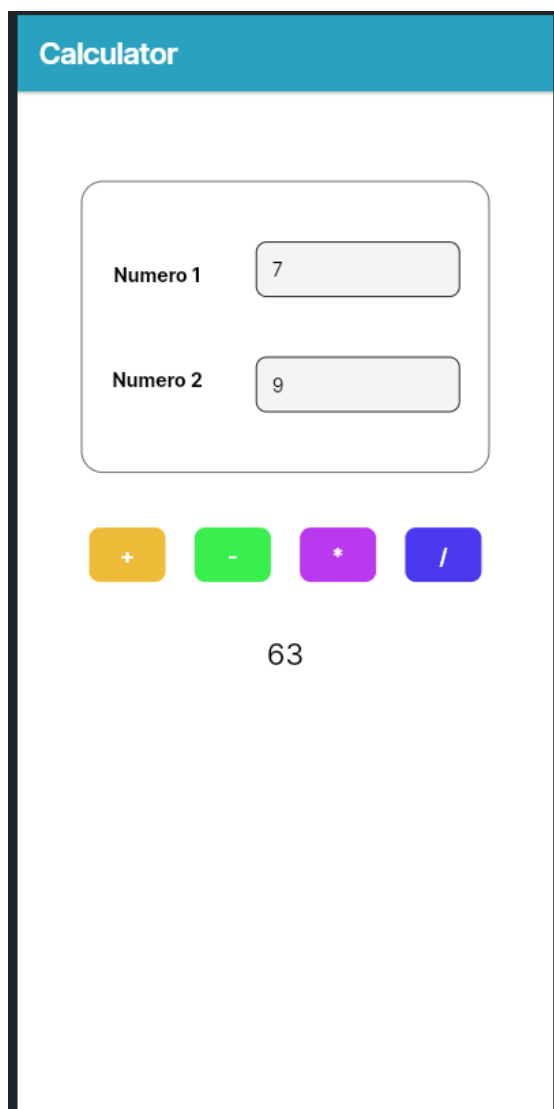
Anexos

The image shows a mobile application interface for a calculator. At the top, there is a blue header with the word "Calculator" in white. Below the header, there is a white rounded rectangle containing two input fields. The first field is labeled "Numero 1" and contains the number "7". The second field is labeled "Numero 2" and contains the number "9". Below these fields, there are four colored buttons: a yellow button with a "+" sign, a green button with a "-" sign, a purple button with a "*" sign, and a blue button with a "/" sign. Below the buttons, the result "16" is displayed.

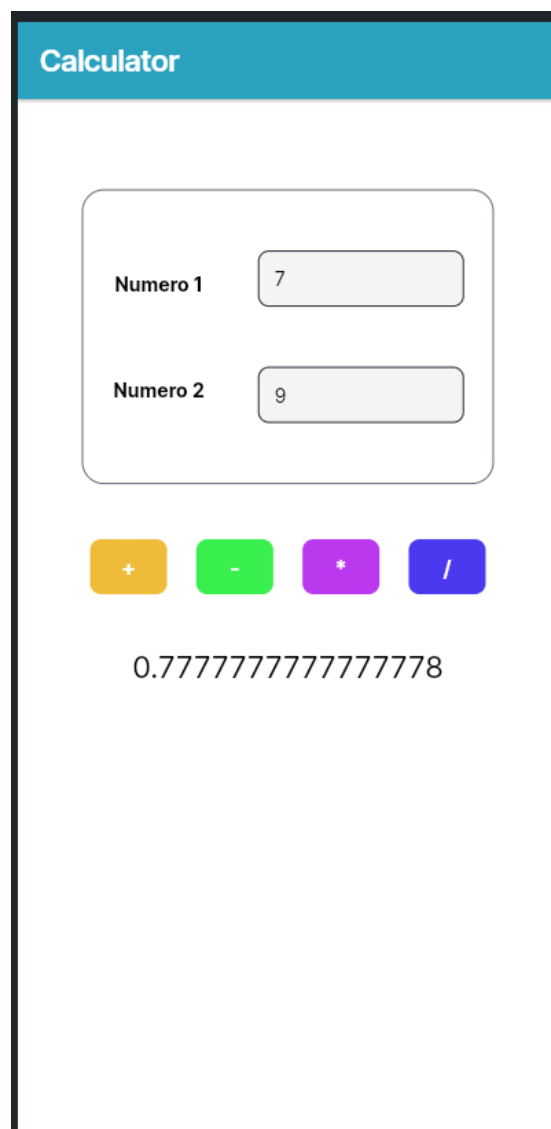
Anexo 1: Operación de suma: $7 + 9 = 16$

The image shows a mobile application interface for a calculator, similar to the one in Anexo 1. It has a blue header with "Calculator" in white. Below the header, there is a white rounded rectangle containing two input fields. The first field is labeled "Numero 1" and contains the number "7". The second field is labeled "Numero 2" and contains the number "9". Below these fields, there are four colored buttons: a yellow button with a "+" sign, a green button with a "-" sign, a purple button with a "*" sign, and a blue button with a "/" sign. Below the buttons, the result "-2" is displayed.

Anexo 2: Operación de resta: $7 - 9 = -2$



Anexo 3: Operación de multiplicación: $7 \times 9 = 63$



Anexo 4: Operación de división: $7 \div 9 \approx 0,7778$